claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_134043cd-8c3\agent-a6e518a1b4435a621.jsonl`
- SHA-256: `e1d4d7c2131f6b2cd899ee3ad9fee706847dc42cea80df12375e99f237fb31d0`
- Source modified: `2026-06-20T09:02:57+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_134043cd-8c3`
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`

Transcript

1. user (2026-06-20T08:59:23.189Z)

IMPORTANT grounding (verified this session): the engine is now DECOUPLED ? pluggable StorageBackend (local + S3/GCS/DO Spaces) in backend/storage/, and a worker `python -m backend.worker {infer|train}` (backend/worker/__main__.py) that pulls from storage -> runs the pipeline -> pushes results. Validated e2e on the CPU droplet 168.144.126.176 reading/writing DO Spaces with the GPU MOCKED (stub detector, RALLY_DETECTOR_IMPL=stub). BOTH the khelsutra site (/srv/khelsutra-site) AND the engine (~/rally) already live on that ONE droplet. The droplet is CPU-only (no GPU) and a SHARED testbed (MinIO 9000/9001, GCS emu 9023) ? do not propose touching those. Real GPU WASB (NativeWasbRunner, "M3.5") is the NEXT build, not done. The FastAPI has NO auth (CORS '*'). EXCLUDE any path under .claude/worktrees/ ? those are stale copies; read canonical docs/ + backend/ only.

Read the DECOUPLED-COMPUTE architecture in the engine repo (cwd = C:\\Users\\avidu\\Projects\\badminton-highlight-indexer). Read: docs/DECOUPLED_COMPUTE/README.md, 02-COMPUTE-MAP-AND-CONTRACT.md, 07-COMPUTE-ABSTRACTION.md, 03-PLATFORM-OPTIONS.md, 04-BUILD-AND-TEST-PLAN.md, and backend/worker/__main__.py + backend/storage/ (the modules). Map the StorageBackend + ComputeBackend contracts, the worker CLI, the bucket-driven flow, what is built vs next (M3.5 native WASB), and crucially HOW A SINGLE-BOX PROFILE maps onto this. Quote path:line. Return the schema.

2. assistant (2026-06-20T08:59:25.682Z)

I'll read the decoupled-compute architecture docs and the relevant modules. Let me start by pulling the repo fresh and reading the key files in parallel.

3. user (2026-06-20T08:59:31.235Z)

fatal: Cannot fast-forward to multiple branches.
---BRANCH---
feat/native-wasb-runner
---HEAD---
5c77f48 docs(decoupled): HANDOFF.md ? session checkpoint / resume-here (#255)

4. user (2026-06-20T08:59:31.532Z)

01-EXISTING-PLANS-AND-GAP.md
02-COMPUTE-MAP-AND-CONTRACT.md
03-PLATFORM-OPTIONS.md
04-BUILD-AND-TEST-PLAN.md
05-COST-ATTRIBUTION.md
06-RISKS-AND-OPEN-QUESTIONS.md
07-COMPUTE-ABSTRACTION.md
HANDOFF.md
README.md
---STORAGE---

```
__init__.py
__pycache__
base.py
gcs.py
gdrive.py
local.py
ref.py
s3.py
---WORKER---
__init__.py
__main__.py
__pycache__
```

5. assistant (2026-06-20T08:59:35.759Z)

```
The branch can't fast-forward (diverged); I'll proceed read-only on the local checkout as
instructed. Let me read the key docs and modules in parallel.
```

6. user (2026-06-20T08:59:37.006Z)

```
1       # Decoupled Compute ? Cloud-Native GPU for Train + Inference
2
3       **Status:** ? PROPOSAL / research ? for owner review. **No code yet.**
4       **Date:** 2026-06-20 · **Owner:** avidullu
5       **Lifecycle:** *Critical active project.* When delivered, **move this whole directory to
`docs/archives/past_projects/`** (keeps the strategy tree clean).
6
7       > **What this directory is.** A single, unified place that answers one question: *can we
lift the
8       > rally-segmentation pipeline off the owner's local Windows/WSL box and run it on a
**rented,
9       > credit-budgeted, cloud-native GPU** ? driven by an API key ? that does **both** (a)
**retrain** the
10      > rally model from a **bucket of videos** and (b) run **inference on new uploads** ? and
**bill each
11      > video / each training run exactly**, including for a non-owner user?* Short answer:
the *design*
12      > already exists across our strategy docs; the *build* (a container/VM image, a real
bucket, a chosen
13      > provider, a cost ledger) does not. This directory inventories what's planned, maps the
real compute,
14      > compares platforms, and lays out a cheapest-first build & test plan that fits the
**$200** budget.
15
16      ---
17
18      ## North Star
19
20      A **decoupled** pipeline where the **control plane** (a thin API that accepts jobs and
reports status)
21      is separated from the **data/compute plane** (the heavy bytes + GPU work), which runs in
a **rented
22      cloud GPU** reading from and writing to an **object-store bucket**. Two verbs:
23
24      - **`train`** ? consume a bucket of videos + labels ? produce versioned model weights
back to the bucket.
25      - **`infer`** ? a new uploaded video ? rally segments + highlight reel + an eval report
back to the bucket.
26
27      ?with **exact per-video (inference) and per-run (training) cost attribution** so the
cost can be
28      charged to whoever ran the job.
29
30      ---
31
```

TL;DR (read this, then dive into the numbered docs)

1. **Yes ? decoupling is already designed, in depth.** [`PLATFORM_ARCHITECTURE.md`](../PLATFORM_ARCHITECTURE.md) is a control-plane/data-plane split with a `ComputeBackend` registry (`local | hosted | byo_worker`) and a `StorageBackend` registry (S3/GCS/Azure via `fsspec`); [`HOSTING_PLAN.md`](../HOSTING_PLAN.md) already names *"GPU box = first `byo_worker`"* with GCS buckets + signed-URL uploads; §5A specifies **training on a rented GPU** via `ms-swift`. ? **[01](01-EXISTING-PLANS-AND-GAP.md)**.
2. **What's missing is everything that turns those seams into the North Star:** a container/VM image, a **chosen** provider + an **API-key/provisioning** flow, a **real bucket** (today it's OneDrive auto-sync ? a proto-bucket), a **download-to-local** shim, path de-hardcoding, a **bucket-triggered retrain**, a **remote inference endpoint**, and **cost metering**. ? **[01](01-EXISTING-PLANS-AND-GAP.md)**, **[02](02-COMPUTE-MAP-AND-CONTRACT.md)**.
3. **$200 is comfortable ? for the *right* scope.** It easily covers the **inference pipeline + Gen-0 head training** (numpy-class, trains in minutes on a small GPU) decoupled onto one cheap GPU. It does **NOT** cover the **Gen-2 VLM fine-tune** (ms-swift/QLoRA, multi-GPU, *"the largest open risk, currently unquantified"*) ? that stays explicitly **out of scope** for this budget. ? **[03](03-PLATFORM-OPTIONS.md)**, **[06](06-RISKS-AND-OPEN-QUESTIONS.md)**.
4. **Docker is optional.** Serverless/Cloud Run paths need an image; a **DO Droplet** or **SkyPilot** path can run the repo in a plain `venv`/`conda` with no container. We recommend a container *where you go serverless* and a VM/SkyPilot path otherwise. ? **[03](03-PLATFORM-OPTIONS.md)**.
5. **Per-video billing falls out of a serverless per-invocation metering model + a cost ledger on the** already-shipped `jobs` table.** ? **[05](05-COST-ATTRIBUTION.md)**.
6. **Cheapest-first: M0 costs $0** ? a CPU container/venv runs the **committed synthetic, video-free regression fixtures** ([`golden_fixtures.py`](../../backend/eval/golden_fixtures.py)) in CI, no GPU, no WSL. Only spend a dollar at M1. ? **[04](04-BUILD-AND-TEST-PLAN.md)**.
7. **The WSL detectors run seamlessly off the Windows box ? and we prove it on DigitalOcean.** The `DetectorRunner` ABC is already built for the swap (WSL plumbing is an isolated mixin; the `Frame, Visibility,X,Y` CSV is the parity contract); a DO GPU is Ubuntu+CUDA = your WSL conda env, **minus WSL**. A dedicated **M3.5 parity test on DO** is the guarantee, and it doubles as the `DESKTOP_APP_PLAN` P0 spike. ? **[02](02-COMPUTE-MAP-AND-CONTRACT.md)**, **[04](04-BUILD-AND-TEST-PLAN.md)**, **[06](06-RISKS-AND-OPEN-QUESTIONS.md)**.

The budget tension you must decide (see [03](03-PLATFORM-OPTIONS.md) + [06](06-RISKS-AND-OPEN-QUESTIONS.md))

You have **$200 in DigitalOcean**, and you're open to **GCS + a Cloud-Run-like** architecture. These pull in different directions for the *side goal* (exact per-video billing):

- **DigitalOcean** has the credits + **Spaces** (S3-compatible, **1 TiB egress included** ? the best video-download deal), but **no true serverless GPU**, so per-video metering is coarser (you bill wall-clock on a started/stopped GPU Droplet).

```
71      - **GCP Cloud Run (GPU) / RunPod Serverless** give **scale-to-zero + exact per-
invocation GPU-seconds**
72        ? which *is* your per-video bill ? but the $200 isn't DO credit there (GCP's
$300/90-day trial is the
73        clean free tier; RunPod's "free credit" is small).
74
75      **Recommended split:** prototype M0?M2 on **DO (use the credits) + DO Spaces**; stand up
the M3 inference
76      *endpoint* where per-invocation metering is native (Cloud Run GPU / RunPod Serverless).
Final call is
77      owner's ? tracked in [06](06-RISKS-AND-OPEN-QUESTIONS.md).
78
79      ---
80
81      ## Contents
82
83      | # | Doc | What it answers |
84      |---|---|---|
85      | **00** | [**HANDOFF ? resume here**](HANDOFF.md) | **? Current status (CPU e2e
working) + the next build (M3.5 native WASB). Read first.** |
86      | 01 | [Existing plans & the gap](01-EXISTING-PLANS-AND-GAP.md) | What we've already
planned for decoupling, and precisely what's unbuilt. |
87      | 02 | [Compute map & data contract](02-COMPUTE-MAP-AND-CONTRACT.md) | Where
GPU/CPU/WSL/API actually live in the 3 repos; what must move to a bucket. |
88      | 03 | [Platform options](03-PLATFORM-OPTIONS.md) | DO vs GCP vs serverless GPU; Docker-
optional alternatives; provider matrix. |
89      | 04 | [Build & test plan](04-BUILD-AND-TEST-PLAN.md) | The two entrypoints, bucket
layout, the M0?M4 milestone ladder, first PR. |
90      | 05 | [Cost attribution](05-COST-ATTRIBUTION.md) | Per-video inference billing + per-
run training cost segmentation for any user. |
91      | 06 | [Risks & open questions](06-RISKS-AND-OPEN-QUESTIONS.md) | The legal/scope/budget
gates and the decisions only the owner can make. |
92      | 07 | [Compute abstraction](07-COMPUTE-ABSTRACTION.md) | How we segment GPU vs CPU work
+ a spec-aware `DeviceContext`/`DeviceProfile` layer (the "GPU Runner"). |
93
94      ---
95
96      ## Verification status (checked against code 2026-06-20)
97
98      These load-bearing facts were confirmed against the current tree (not just the older
strategy docs):
99
100     - ? **Async-job control plane shipped** ? `jobs` table at
[`database.py:188`](../../backend/infrastructure/database.py), worker at
[`api/job_worker.py`](../../backend/api/job_worker.py), `/api/process` ? 202 + `job_id`.
101     - ? **`jobs` table has no cost/tenant columns yet** (cols: id, video_path, segmenter,
player_pool, max_frames, status, policy, result, progress, timings) ? so the cost-ledger delta
in [05](05-COST-ATTRIBUTION.md) is net-new.
102     - ? **Gen-0 harness exists** ?
[`training/gen0/harness.py`](../../training/gen0/harness.py) (one-command train?eval?gate).
103     - ? **Committed synthetic, video-free regression fixtures exist** ?
[`backend/eval/golden_fixtures.py`](../../backend/eval/golden_fixtures.py) ? M0 is a genuine $0
CI gate.
104     - ? **`"output"` path still hardcoded** ?
[`trajectory_hybrid.py:196,272`](../../backend/pipeline/segmenters/trajectory_hybrid.py) (a real
bucket-relocation blocker).
105     - ? **No Dockerfile anywhere** in any of the three repos ? the container is greenfield.
106     - ? **Collector keys by human `video_id`, not MD5** ? `curate.py`; the export manifest
omits an `md5` field, which the MD5-keyed consumer needs (see [02](02-COMPUTE-MAP-AND-
CONTRACT.md)).
107
108     ## Provenance
109
110     Built from a 13-agent research workflow (?970k tokens, 5 plan-harvest + 3 compute-map +
2 cloud-provider
```

```
111     agents, then synthesis + an adversarial completeness critic), cross-checked against the
owner's strategy
112     docs and the live code. Cost/price figures are **approximate ? verify current provider
terms before
113     committing budget.**
114
```

7. user (2026-06-20T08:59:37.950Z)

```
1       # 02 · Compute map & data contract
2
3       **Status:** research synthesis, code-verified 2026-06-20 · **Date:** 2026-06-20
4
5       > What actually consumes GPU/CPU/WSL/external-API across the three repos, ranked by how
hard each is to
6       > lift into a Linux cloud-GPU container ? plus the one data contract everything keys on.
7
8       ## 1. Compute footprint of `badminton-highlight-indexer` (the consumer/trainer)
9
10      | Component | Needs | Portability risk ? cloud Linux GPU |
11      |---|---|---|
12      | **`gemini` segmenter** (default) | external paid API (`GEMINI_API_KEY`) | **LOW.**
Pure HTTP; no GPU. Silently returns **zero** segments without the key ? inject as a secret. |
13      | **`motion` segmenter** | CPU (OpenCV frame-diff) | **NONE.** Pure CPU, no API key. |
14      | **`yolo_hybrid` segmenter** | GPU (torchvision Faster R-CNN + ByteTrack); auto CPU-
fallback | **LOW.** Pure-pip; runs on CPU or CUDA. Pin `supervision<0.30` (ByteTrack API
drifts). Pre-bake COCO weights into the image to avoid first-run hub pulls. |
15      | **`trajectory_hybrid` / TrackNet / WASB** | **GPU + WSL2 bridge** (TensorFlow);
hardcoded `~/models` paths, hardcoded `device='cuda'` (no CPU fallback); **un-vendored**
research repos; TrackNet healthcheck requires a visible GPU | **HIGH ? but the swap seam is
already built.** Runs *only* via `wsl -d Ubuntu ? conda activate` **on Windows** ? on a Linux
GPU box WSL is **dropped, not ported** (the conda env runs natively). The `DetectorRunner` ABC
([`detectors/base.py`](../../backend/pipeline/detectors/base.py)) exists *expressly* to de-risk
this: WSL plumbing is a separate `WslCommandMixin` (a native runner must not inherit it),
Windows behavior is *"100% preserved,"* and the **parity contract is the `Frame,Visibility,X,Y`
CSV** ? any runner emitting it reuses `parse_trajectory_csv` + `trajectory_to_action_windows`
verbatim, so everything downstream is byte-identical. The `LinuxNativeRunner` impl doesn't exist
yet / has never been run ? prove it via the **M3.5 parity test on DigitalOcean** ([04](04-BUILD-
AND-TEST-PLAN.md)). |
16      | **`fusion_hybrid` / Gen-0 head** | CPU/tiny GPU (numpy/scipy) | **NONE.**
[`training/gen0/harness.py`](../../training/gen0/harness.py): ~1M params, ~4.5 GB VRAM, trains
in *minutes*. This is the cheap retrain target. |
17      | **`ffmpeg` chunking / stitching** | CPU (`libx264`) | **LOW.** No nvenc/driver-codec
dependency (a portability win) ? `apt-get install ffmpeg` is enough. |
18      | **Gen-2 VLM** (ms-swift / QLoRA Qwen3-VL) | **multi-GPU** (6?8-GPU GRPO reference; 24
GB VRAM floor) | **OUT OF SCOPE for $200.** *"Largest open risk, currently unquantified."* Do
not attempt on this budget. |
19
20      **Two structural container blockers beyond WSL:**
21      - **TF-vs-torch runtime conflict.** TrackNet is TensorFlow; the rest is PyTorch. Their
pip-bundled CUDA
22        userspaces disagree ? *"works alone, segfaults together."* Fix = **two single-
framework images** (a torch
23        image, a TF image) sharing a `/scratch` volume. So the container topology itself is
non-trivial.
24      - **CUDA wheel selection.** `torch>=2.12.0` pinned with no index ? the
Dockerfile/installer must pick
25        `--index-url ?/cu124` vs `?/cpu` explicitly.
26
27      **Path / I/O blockers (confirmed live):**
28      - Hardcoded literal `"output"` in
[`trajectory_hybrid.py:196,272`](../../backend/pipeline/segmenters/trajectory_hybrid.py) ? not
redirected by `--output-dir`.
29      - Local-file assumptions: `cv2.VideoCapture`, `ffmpeg -i <path>`, whole-file MD5,
`allowed_video_root`
```

```
30        check in [`main.py`](../../backend/main.py). No URL/stream path ? needs a
`fetch(uri)?local` + `put` shim.
31
32      ## 2. Compute footprint of `sports-data-collector` (the producer/curator)
33
34      CPU-first; *"the gap is narrow."* Only GPU touchpoint is **opt-in `nvenc`** (`--encoder
nvenc`, commit #28)
35      for proxy transcode + batch transcode ? defaults to CPU `libx264`. Shares the same
local-filesystem + MD5
36      assumptions. The export **`manifest.json` omits an `md5` field** (`proxy.py` carries
only `source_md5` /
37      `proxy_md5`); the collector keys by **human `video_id`** (`curate.py`), while the
indexer keys by **file
38      MD5** ? see §4.
39
40      ## 3. Compute footprint of `rally-annotator` (the labeler)
41
42      Effectively **no heavy compute** ? a VLC 3.x Lua extension a human drives to mark rally
`start`/`end` +
43      ending reason ? `<video>.rallies.csv`. Nothing to containerize; it's the *source* of the
labels that must
44      land in the bucket.
45
46      ## 4. The one data contract everything keys on
47
48      ```
49       rally-annotator            sports-data-collector            badminton-highlight-indexer
50       (label by hand) ?????????? (curate + license + export) ????
(validate?segment?index?stitch?eval?train)
51       <video>.rallies.csv        SQLite ? manifest.json + CSVs     keyed by file MD5
52      ```
53
54      - **Identity:** the indexer keys videos by **whole-file MD5** (`compute_video_id`, 64
KB-chunked); the
55        collector keys by **human `video_id`**. The manifest's `local_path` ties them
together. **Any re-encode
56        changes the MD5** and silently breaks the join.
57      - **Intervals:** temporal `[start, end)` in **decimal seconds**; ending-reason taxonomy
58        (winner/forced_error/unforced_error/service_fault/let/other) is the cross-sport label
set.
59      - **Train/serve wall (ADR-008):** the boundary between training and serving is an
**artifact bundle**
60        (weights + manifest), **never a code import** ? keyed by content/semver.
61
62      ### What this means for a bucket workflow
63
64      1. **Add `md5` to the collector export manifest** (or decide *original-vs-proxy* keying)
**before** feeding
65         a bucket ? otherwise the MD5-keyed consumer can't resolve labels?video. *This is an
M2 prerequisite, not
66         a footnote.*
67      2. **Store byte-identical files** the labels were made against. If you must proxy-
transcode for size, carry
68         an explicit `original_md5 ? proxy_md5` mapping so the join survives.
69      3. A cloud `train`/`infer` job's **inputs** = `{video object(s), label CSV(s),
manifest.json}` from the
70         bucket; its **outputs** = `{versioned weight bundle}` (train) or `{highlights.mp4,
intervals.csv,
71         report.json}` keyed by `video_id` (infer). These map cleanly onto the existing per-
video output layout.
72
73      ### Inputs/outputs a cloud job must exchange (summary)
74
75      | Verb | Pulls from bucket | Pushes to bucket |
76      |---|---|---|
```

77 | `train` | `videos/`, `labels/`, `manifest.json` (+ latest checkpoint to resume) |
`weights/<semver>/{model, manifest}` + periodic `weights/checkpoints/` |
78 | `infer` | one `videos/<md5>.mp4` | `outputs/<video_id>/{highlights.mp4, intervals.csv,
report.json}` |
79

8. user (2026-06-20T08:59:38.305Z)

1   # 07 · Compute abstraction ? segmenting GPU vs CPU + a spec-aware device layer
2
3   **Status:** PROPOSAL (design) · **Date:** 2026-06-20 · code-verified 2026-06-20
4
5   > *"Do we have a GPU Runner interface? How do we segment GPU vs CPU work ? cleanly
abstracting the GPU while
6   > leaving room for customizations based on the GPU spec?"* Answer: **not yet as a single
interface**, but the
7   > seam and a spec-probe already exist. This doc defines the missing **device layer** and
the **GPU/CPU
8   > segmentation rule** ? small, and consistent with the codebase's ABC+Registry idiom.
9
10   ## 1. What exists today (honest inventory)
11
12 | Concern | Today | Verdict |
13 |---|---|---|
14 | Global GPU switch | `use_gpu: bool = True`
([`config/models.py:119`](../../backend/config/models.py)) | Coarse ? one flag for the whole
run. |
15 | torchvision person detector | `self.device = "cuda" if use_gpu and
torch.cuda.is_available() else "cpu"` + warn/fallback
([`person_detector.py:74-76`](../../backend/pipeline/detectors/person_detector.py)) | **Clean
fallback, but no spec-awareness** (no VRAM/batch/precision logic; per-frame `.to(device)`). |
16 | WASB detector | hardcoded `device='cuda'`, **no CPU fallback** | Inconsistent ? a
portability/robustness gap. |
17 | TrackNet (TF) | GPU only via WSL; healthcheck requires a visible GPU | WSL-bound (see
[02](02-COMPUTE-MAP-AND-CONTRACT.md), [04 M3.5](04-BUILD-AND-TEST-PLAN.md)). |
18 | `DetectorRunner` ABC | abstracts the **task** (run_predict/healthcheck)
([`detectors/base.py`](../../backend/pipeline/detectors/base.py)) | Device handling lives
inside each concrete runner, not shared. |
19 | `ComputeBackend` (planned) | abstracts **where** a job runs (local/hosted/byo_worker)
| Orthogonal to *how* GPU work executes ? don't conflate. |
20 | GPU spec probe | `gpu_name()` + `gpu_stats()` (name, VRAM used/free via `nvidia-smi`)
([`run_telemetry.py`](../../backend/utils/run_telemetry.py)) | **The seed of a `DeviceSpec` is
already here.** |
21 | Macro GPU/CPU split | detect-once-on-GPU ? persist CSV/features ? re-score on CPU
(EXPERIMENT_HARNESS, GOLDEN_DATA) | **Already the right segmentation ? at the artifact
boundary.** |
22
23   **Conclusion:** there is no unified "GPU Runner." Device selection is scattered across
components with three
24   different styles (global flag, clean-fallback, hardcoded-cuda). The fix is one small
shared layer.
25
26   ## 2. Two orthogonal axes ? keep them separate
27
28   ```
29   WHERE the job runs   ?   HOW a GPU stage executes on this box
30   (ComputeBackend ? coarse,   ?   (DeviceContext ? fine, spec-aware,
31   per-job: local | DO droplet ?   per-stage: probe the actual GPU ?
32 | serverless | byo_worker)   ?   resolve batch/precision/chunk knobs)
33   picks the machine ???????? on that machine, DeviceContext.detect()
34   ?   probes the real GPU and tunes the work
35   ```
36
37   `ComputeBackend` answers *which machine*; **`DeviceContext` answers *given this
machine's GPU, how do I run

this stage*.** A serverless L4, a DO RTX-4090 Droplet, and CPU CI each get correct knobs **automatically**,
with per-spec overrides available. Conflating the two is the main design trap.

3. The GPU/CPU segmentation rule

Classify every pipeline stage; GPU work is concentrated in the detection substrate.

Stage	Class	Notes
Validators (format/res/pts/scene/blur/relevance)	**CPU**	ffprobe + OpenCV/numpy
ffmpeg chunk extraction / stitching	**CPU** (`libx264`); GPU optional (`nvenc`)	no driver-codec dependency by default
Person detection (torchvision Faster R-CNN)	**GPU** (auto CPU-fallback)	`DeviceContext` consumer
ByteTrack tracking	**CPU**	runs on the detections
Shuttle trajectory (WASB torch / TrackNet TF)	**GPU**	`DeviceContext` consumer; the WSL?native target
Windowing / candidate merge	**CPU** (numpy)	model-agnostic
AI handoff (Gemini)	**network I/O**	no local GPU; `ai_max_workers` parallelism
DB writes, eval/scoring, calibration	**CPU** (numpy)	device-agnostic
Gen-0 head training	**tiny GPU/CPU**	~4.5 GB, minutes

The rule: *a GPU-bound stage is a pure, restartable unit whose only job is `frames ? artifact` (the
`Frame,Visibility,X,Y` CSV or a feature JSON). Everything downstream is device-agnostic CPU that reads that
artifact.* This single principle delivers three things at once:
1. **Clean GPU/CPU segmentation** ? only the detection substrate touches a `DeviceContext`.
2. **Cross-machine decoupling** (the North Star) ? the artifact boundary is exactly where GPU and CPU work
   can live on *different machines* (it's already how EXPERIMENT_HARNESS / GOLDEN_DATA split today).
3. **Per-stage billing** ([05](05-COST-ATTRIBUTION.md)) ? GPU-seconds are attributable to the GPU stage.

4. The proposed device layer (the "GPU Runner")

Two dataclasses + one context object ? injected into GPU stages, **replacing** scattered `device=` logic.

```python
@dataclass(frozen=True)
class DeviceSpec:                       # WHAT hardware we got (probe once, cache)
    kind: str                           # "cuda" | "mps" | "cpu"
    name: str | None                    # "NVIDIA L4" ? from run_telemetry.gpu_name()
    vram_gb: float | None               # from gpu_stats() memory.free/used
    compute_capability: tuple[int,int] | None
    supports_fp16: bool
    supports_bf16: bool
    num_devices: int

@dataclass
class DeviceKnobs:                      # HOW to run, resolved from spec (§5)
    batch_size: int
    precision: str                      # "fp32" | "fp16" | "bf16"
    chunk_duration_s: float
    frame_skip: int
    max_inflight_chunks: int
    ai_max_workers: int

class DeviceContext:                    # the GPU Runner ? injected into GPU stages
    spec: DeviceSpec
    knobs: DeviceKnobs
```

```
92          def to(self, model_or_tensor): ...           # placement (cuda/mps/cpu)
93          def autocast(self): ...                       # precision ctx (no-op on cpu)
94          def run_with_oom_retry(self, fn, halve="batch_size"): ...   # CUDA-OOM ? smaller
batch
95          @classmethod
96          def detect(cls, config) -> "DeviceContext":   # probe spec ? resolve profile (§5)
97              ...
98      ```
99
100     - **One probe, reused:** `DeviceSpec` is built from the `nvidia-smi` data
`run_telemetry` already collects ?
101       so the spec also feeds the **cost ledger's `gpu_type`** ([05](05-COST-ATTRIBUTION.md))
and the run-log
102       header. One source of truth for "what GPU am I on."
103     - **Graceful by construction:** no CUDA ? a `cpu` spec + conservative knobs. This is
where **WASB's
104       hardcoded `device='cuda'` gets fixed** ? it becomes `ctx.to(model)`.
105     - **`DetectorRunner` keeps its task contract;** the device is injected (constructor or
call), so the
106       `LinuxNativeRunner` ([M3.5](04-BUILD-AND-TEST-PLAN.md)) is just *"a `DetectorRunner`
whose `DeviceContext`
107       has a native CUDA spec"* ? the same abstraction serves WSL, native-Linux, and CPU.
108
109     ## 5. Spec-aware customization ? a `DeviceProfile` registry
110
111     The "leave scope for customizations based on the GPU spec" requirement = a registry
(same ABC+Registry idiom
112     as segmenters/providers/validators) that maps a `DeviceSpec` ? tuned `DeviceKnobs`.
113
114     ```python
115     class DeviceProfile(ABC):
116         name: str
117         def matches(self, spec: DeviceSpec) -> bool: ...
118         def knobs(self, spec: DeviceSpec, overrides: dict) -> DeviceKnobs: ...
119
120     # Built-ins (illustrative): CpuProfile, T4_16GB, L4_24GB, A4000_16GB, RTX4090_24GB,
A100_40GB ?
121     # + HeuristicProfile: derive batch_size/precision from vram_gb + compute_capability when
the
122     #   exact GPU name isn't in the table (so an unknown GPU still runs sensibly).
123     ```
124
125     **Resolution order (most specific wins):**
126     1. **Explicit override** ? `config.indexing.device.{batch_size,precision,?}` or a CLI
flag (`--batch`, `--precision`). *Full manual control.*
127     2. **Matched named profile** ? e.g. an `L4_24GB` profile (bf16, batch N, chunk T).
128     3. **Heuristic from `DeviceSpec`** ? derive from VRAM + compute capability when the name
is unknown.
129     4. **Conservative CPU defaults** ? when there's no GPU.
130
131     So the GPU is **cleanly abstracted** (stages call `ctx.to()/ctx.autocast()/ctx.knobs`,
never `torch.cuda`
132     directly) **and** customizable per spec (a config override tunes batch/precision/chunk
for a 4090 vs an L4).
133
134     > **Scoping (owner decision 2026-06-20): a small *provider-grounded* set + heuristic
fallback.** Curate
135     > profiles only for GPUs your providers actually rent, and fall back to a **VRAM
heuristic** for everything
136     > else. This flips the earlier YAGNI worry for one reason: **the cost ledger
([05](05-COST-ATTRIBUTION.md))
137     > needs the *same* `{provider, gpu_type} ? $/hr` table for billing anyway**, so the set
isn't speculative ?
138     > billing requires it and the device knobs ride along on the same rows. The provider-
grounded set:
```

```
139     >
140     > | Tier | GPUs | Notes |
141     > |---|---|---|
142     > | **DO / Paperspace (now)** | **M4000 8GB (free tier)** · P4000/RTX4000 · A4000 16GB ·
A5000/A6000 · L40S · A100 | M4000 is the unbilled Gradient-plan card (Maxwell `sm_52` ? verify
torch build); credit covers the rest |
143     > | **GCP (later)** | T4 16GB · L4 24GB (Cloud Run GPU) · A100 | T4/L4 are GCP/AWS,
**not** DO |
144     > | **RunPod/Vast (only if used)** | RTX 4090 24GB | **4090 is not on DO or GCP** ? add
only if serverless-on-RunPod |
145     > | **fallback** | anything else | `HeuristicProfile` derives knobs from VRAM + compute
capability |
146     >
147     > Two caveats: (1) a named entry exists for **identity + $/hr rate first**, tuning
second ? keep its perf
148     > knobs **= the heuristic until a benchmark shows a per-GPU win** (the workload is
inference-light, batch?1,
149     > so tuning may never matter). (2) `DeviceContext` itself is **not urgent**: WASB's
hardcoded
150     > `device='cuda'` only bites when the native runner lands, so **build it at
[M3.5](04-BUILD-AND-TEST-PLAN.md),
151     > not at M0.**
152
153     ## 6. Smallest first step (do this at M3.5, not M0)
154
155     1. Add `DeviceSpec` + `DeviceContext.detect()` from the existing `run_telemetry` probes;
ship a
156        `CpuProfile` + a `HeuristicProfile` (derive batch/precision from VRAM) **+ a
config/CLI override**, plus
157        the small **provider-grounded** set (DO/Paperspace now, GCP later ? §5) whose tuning
knobs **start equal
158        to the heuristic** (they exist for identity + the cost-ledger rate, not tuning, until
measured). Defaults
159        reproduce today's behavior (no-op until adopted).
160     2. Route **WASB** through it when you build the native runner ([M3.5](04-BUILD-AND-TEST-
PLAN.md)) ? this is
161        the consumer that justifies the layer (it removes the hardcoded-`cuda` footgun). The
torchvision person
162        detector can follow (it already has clean fallback, so it's optional cleanup).
163     3. Feed `spec.name`/`spec.vram_gb` into the cost ledger's `gpu_type` ([05](05-COST-
ATTRIBUTION.md)) ? one
164        probe, two uses.
165
166     This lands the unified GPU abstraction without touching the windowing/eval/CPU paths ?
they stay
167     device-agnostic by design (§3). **Add named `DeviceProfile`s only when a benchmark shows
per-GPU tuning
168     pays.**
169
```

9. assistant (2026-06-20T08:59:40.988Z)

Now let me read the HANDOFF, build/test plan, platform options, and the actual modules in parallel.

10. user (2026-06-20T08:59:41.945Z)

```
1       # DECOUPLED_COMPUTE ? Session Handoff / Resume-Here
2
3       **Updated:** 2026-06-20 · **Read this first to resume.** Companion: this whole directory
+ `docs/SETUP_NEW_MACHINE.md`.
4
5       ## TL;DR ? where we are
6
7       The **entire bucket-driven `train ? infer` pipeline runs end-to-end on a CPU box**, with
```

the GPU **mocked**
(deterministic stub). Validated on a DigitalOcean droplet reading/writing **DO Spaces**, driven remotely over
SSH. Everything is merged to `master` (PRs #246?#254); the suite is **936 green**.

The ONE remaining piece for *real* (non-mock) training is the **Linux-native WASB `DetectorRunner` (M3.5)**
running on a **GPU box** ? it produces *real* shuttle trajectories instead of the synthetic stub. Once it
exists, the **same `worker train` / `worker infer` commands run unchanged**; only the trajectory source flips.

What's built (all merged to master)

PR	What	Key files
#249	Storage layer ? `local` + **S3 (AWS/DO Spaces/R2/MinIO)** + GCS, URI-driven; gdrive stub	`backend/storage/`
#251, #254	**Worker** `python -m backend.worker {infer|train}` ? pull from storage ? pipeline ? push	`backend/worker/__main__.py`
#247	CPU-mock detector seam (`RALLY_DETECTOR_IMPL=stub` / `indexing.detector_impl=stub`)	`backend/pipeline/detectors/stub_runner.py`
#253	GPU provisioning ? NVMe scratch + **native WASB setup** + port runbook	`deploy/digitalocean/{bootstrap,mount_scratch,gpu_setup}.sh`
#252	Cross-platform canonicalization (Linux/macOS, not just Windows)	(segmenters, compiler, doctor.py)
#246	The plan directory	`docs/DECOUPLED_COMPUTE/`

Verified e2e (CPU, no GPU, no Gemini key): `worker train` pulled 7 labels from Spaces ? `synth_trajectory_from_labels`
(label-consistent CPU mock) ? `training.gen0.harness` (shelled out ? `backend` must not import `training`) ?
LOVO over 7 videos (F1 1.000 on the mock, gate=no-ship by design) ? `weights/` to Spaces. `worker infer` pulled a
real video ? stub ? `outputs/<md5>/` to Spaces.

Resources & access (current)

- **CPU droplet:** `168.144.126.176` (region `blr1`, CPU-only, no NVMe). SSH: `ssh -i ~/.ssh/do_rally root@?`
  (key generated locally; pubkey added to DO). Code + venv at `~/rally`; Gemini key at `/root/.keys/GEMINI_API_KEY`.
- **DO Spaces:** bucket **`rally-e2e`**, region **`sgp1`**, endpoint `https://sgp1.digitaloceanspaces.com`.
  ? **The Spaces key was exposed in chat ? ROTATE it.** Creds live on the box at `~/.aws/credentials` (never in repo).
- **Corpus staged in Spaces:** `videos/` (8 .mp4) + `labels/` (7 `.rallies.csv`) + `weights/0.1.0-smoke/` + `outputs/`.
- **Drive corpus** (source): folder `1sHGL7iacnmM80ChT-p2cKy638gK504T_` (gdown-able by file-id). 7 `[Done]` = video+labels;
  Video 1 `GX010135_proxy.mp4` is the held-out (non-Done) test.
- **Helper scripts on the box:** `/root/do_e2e.sh` (reproduces the train+infer e2e), `/root/do_spaces_config.json`
  (`storage.s3` sgp1 + `skip_ai_handoff` + `detector_impl=stub` + permissive validation), `/root/do_stage_{labels,videos}.sh`.

Reproduce the working e2e (today, CPU)

```bash
ssh -i ~/.ssh/do_rally root@168.144.126.176 'bash ~/do_e2e.sh'
# train (7-video LOVO) + infer (real video) from Spaces ? artifacts back to Spaces.
```

NEXT BUILD ? real trajectories (M3.5)

```
52      1. **GPU box** (owner): spin up a DO GPU Droplet **in `sgp1`** (co-located with the
bucket) ? `SETUP_NEW_MACHINE.md` §2
53         has the `doctl` command (verify slugs). Provide a DO API token if you want it
`doctl`-provisioned. Add the same SSH key.
54      2. **Port** (`SETUP_NEW_MACHINE.md` §9): git-archive ? scp ? `mount_scratch.sh` ?
`bootstrap.sh --gpu` ? `gpu_setup.sh`
55         (native WASB conda env) ? secrets (rotated Gemini key + Spaces creds).
56      3. **Build `NativeWasbRunner(DetectorRunner)`** in `backend/pipeline/detectors/` ?
sibling to `WasbRunner` but **without**
57         `WslCommandMixin`: run WASB in-process/native (no `wsl`/`conda
activate`/`to_wsl_mnt_path`), device `cuda/mps/cpu`,
58         weights via env (not `~/models`), emit the identical `Frame,Visibility,X,Y` CSV.
Select it via the existing
59         `detector_impl` seam (`trajectory_hybrid._resolve_runner`). Scout details: `06-RISKS-
AND-OPEN-QUESTIONS.md` (E1?E6 in the
60         merged workflow plan) + `docs/TRACKNET_WSL_SETUP.md`.
61      4. **Acceptance gate:** byte-parity ? same clip ? WSL CSV (Windows) == native CSV
(Linux) + identical candidate windows.
62      5. **Wire real extraction into `worker train`/`infer`:** today `worker train` uses
`synth_trajectory_from_labels`; add a
63         real-extract path that runs the detector per video ? trajectory CSV ? manifest. The
harness + push are unchanged.
64
65      ## Gotchas / open items
66
67      - **`backend` ? `training`** one-way wall ? the worker **shells out** to `python -m
training.gen0.harness`.
68      - **Gen-0 ship-gate F1@tIoU is provisional (#172)** ? advisory; do **not** hard-gate on
it.
69      - **WASB-C3** weight provenance is an open **legal** item before sharing any
trajectories/model externally.
70      - WASB (torch 1.11) and TrackNet (TF) must stay in **separate conda envs** (segfault if
co-installed).
71      - The box still has MinIO (`:9000/:9001`) + a GCS emulator (`:9023`) running from
earlier provider tests (harmless).
72      - Two-provider goal (GCS in a Bangalore-ish region, parallel hosting) and the
`/srv/khelsutra-site` web-hosting verify
73        + a real-Gemini-detection smoke are **queued, not done**.
74
```

11. user (2026-06-20T08:59:42.426Z)

```
1       # 04 · Build & test plan
2
3       **Status:** PROPOSAL · **Date:** 2026-06-20
4
5       > A staged plan a solo dev can stop at any milestone with a working, tested artifact. It
ships the
6       > **container-clean path first** (Gemini + torchvision + motion + Gen-0) and defers the
WSL-bound
7       > WASB/TrackNet detectors until a Linux-native runner exists ? those two are the only
HIGH-risk components
8       > in the stack (see [02](02-COMPUTE-MAP-AND-CONTRACT.md)).
9
10      ## 1. Target architecture (diagram-in-words)
11
12      ```
13         CONTROL PLANE (thin, cheap; already shipped as the async job model)
14         - validates API key, accepts {job_type, uri}; returns 202 + job_id; GET /jobs/{id}
15         - holds NO video bytes; enqueues, polls
16                      ? job descriptor (bucket keys)         ? status / cost
17                      ?                                       ?
18         COMPUTE WORKER (rented GPU; container OR venv; pay GPU-seconds)
19         - ENTRYPOINT dispatcher: mode ? {train, infer}
20         - sync-first: rclone cp bucket ? /scratch (local NVMe); decode/detect on GPU
```

```
        - write artifacts ? /scratch ? upload to bucket; checkpoint train ?30 min
        - record usage (gpu_seconds, egress, gemini_calls) ? cost ledger (05)
                    ? pull inputs                          ? push outputs
                    ?                                      ?
        OBJECT-STORE BUCKET (data plane ? bytes never via the control plane)
        videos/   labels/   manifest.json   weights/   outputs/
```

The control plane coordinates **metadata + jobs**; the **heavy bytes + GPU work live in the worker**; the
bucket is the only shared state. This is the control/data-plane split [`PLATFORM_ARCHITECTURE.md`](../PLATFORM_ARCHITECTURE.md)
already names ? what's new is the container/VM image, the real bucket, the rented GPU, and the cost ledger.

2. The two entrypoints (one dispatcher; container or `run.sh` on a VM)

`train` ? *bucket of videos + labels ? versioned weights back to the bucket:*
1. Look for the latest checkpoint under `weights/checkpoints/`; **resume if present** (spot-safety +
   the firm idempotency/resumability requirement ? a 40-min job must not restart on a blip).
2. `rclone copy` the corpus to `/scratch` (sync-first).
3. Run the **Gen-0 harness as-is** ? already a single self-contained command producing a versioned
   artifact: `python -m training.gen0.harness --manifest <?> --floor <?> --version <semver>`. Gen-0 needs
   only numpy/scipy (~4.5 GB VRAM, minutes). **Gen-2 VLM is explicitly out of scope** ([06](06-RISKS-AND-OPEN-QUESTIONS.md)).
4. Run the product-side **ship-gate as pre-flight** ([`backend/eval/regression.py`](../../backend/eval/regression.py)
   + `metrics.match_intervals`): no model admits itself to serving.
5. Upload weights + manifest to `weights/<semver>/` (ADR-008 artifact bundle, content/semver-keyed).
   Checkpoint every ?30 min.

`infer` ? *new upload ? segments/highlights ? outputs back to the bucket:*
1. `rclone copyto $VIDEO_URI /scratch/in.mp4` ? **sync-first, not a FUSE random-access decode.**
2. Compute whole-file MD5 ? `video_id` (existing `compute_video_id`). **Store the byte-identical file the
   labels were made against** ? any re-encode changes the MD5 and breaks the join ([02](02-COMPUTE-MAP-AND-CONTRACT.md)).
3. Run the segmenter (`gemini` default; `yolo_hybrid`/`motion` offline). Gemini path uploads ?500 MB chunks.
4. Write `outputs/<video_id>/{highlights.mp4, intervals.csv (decimal-second [start,end)), report.json}`.
5. Inference is checkpoint-free ? on spot preemption just retry; make the front-end idempotent.

3. Bucket layout

```
rally-bucket/
??? videos/<md5>.mp4              # immutable, byte-identical to what labels were made on
??? labels/<sport>/<video_id>.csv # collector-exported start,end CSVs
??? manifest.json                 # the join table (video_id, csv, local_path, md5, license, fps?)
??? weights/
?   ??? <semver>/model + manifest # train output (artifact bundle)
?   ??? checkpoints/              # spot-resume state (model + optimizer + step)
??? outputs/<video_id>/{highlights.mp4, intervals.csv, report.json}
```

Auth via env/secret (`AWS_ACCESS_KEY_ID/SECRET` for S3/R2/Spaces, or a mounted GCS

service-account JSON) ?
never baked into the image. Keep `is_commercial`/CURATED training rows physically separated; only those
export by default.

4. Milestone ladder (each with a concrete acceptance test)

M0 ? Local CPU smoke test (`$0`, no GPU, no provider).
Build the torch image *or* a venv; run the **committed synthetic, video-free regression fixtures**
([`golden_fixtures.py`](../../backend/eval/golden_fixtures.py)) + motion/yolo_hybrid on a tiny sample,
CPU-only (torchvision auto-falls-back).
Accept: `infer --video samples/clip.mp4` produces an `intervals.csv`, and the in-repo regression subset
passes ? no GPU, no WSL. (Runnable in GitHub Actions.)

M1 ? Single GPU, manual run.
Launch one GPU (DO Paperspace/Gradient A4000 or GCE Spot L4); run the worker.
Accept: `nvidia-smi` + `torch.cuda.is_available()==True` inside the worker, and one `infer` on a real
clip completes on GPU, faster than the M0 CPU run.

M2 ? Train-from-bucket.
Create the bucket; populate `videos/`, `labels/`, `manifest.json` (**incl. the `md5` field ? prerequisite,
[02](02-COMPUTE-MAP-AND-CONTRACT.md) §4**). Run `train`, pulling via `rclone`, running Gen-0, pushing a
versioned bundle back.
Accept: `weights/<semver>/` appears in the bucket, the artifact passes the ship-gate pre-flight, and a
re-run **resumes from checkpoint** rather than restarting.

M3 ? Inference endpoint on new upload (+ per-video billing).
Deploy as a serverless endpoint (Cloud Run L4 / RunPod Serverless). Control plane returns `202 + job_id`;
worker pulls the upload, writes `outputs/<video_id>/`, and records the cost row ([05](05-COST-ATTRIBUTION.md)).
Accept: `POST {job_type:"infer", video_uri:"?"}` with a valid API key returns a `job_id`; polling yields
`highlights.mp4` + `intervals.csv` in the bucket **and a per-video cost**; a bad/absent key is rejected.

M3.5 ? WSL?native detector parity, tested on DigitalOcean (the "WSL runs seamlessly" gate).
Stand up a **`LinuxNativeRunner`** behind the existing `DetectorRunner` ABC ([`detectors/base.py`](../../backend/pipeline/detectors/base.py)
? *"the main de-risk goal"*): it **skips all `wsl`/`conda activate`/staging**, loads the WASB/TrackNet weights
natively (torch/TF), and writes the **identical `Frame,Visibility,X,Y` CSV** so the model-agnostic
`parse_trajectory_csv` + `trajectory_to_action_windows` (and everything downstream ? windowing, candidates,
AI handoff, eval) run **verbatim, byte-for-byte**. On a **DO GPU Droplet / Paperspace GPU** (Ubuntu +
NVIDIA + CUDA) you replicate the `TRACKNET_WSL_SETUP.md` conda env **natively, minus WSL** ? and since WASB
ships pretrained badminton weights (`wasb_badminton_best.pth.tar`), you get a **real trajectory on DO
today**, no TrackNet training needed. Local Windows dev is **untouched** ? the WSL adapters stay as one
backend of the ABC (the base guarantees *"existing behavior is 100% preserved"*); env/OS selects the runner.
This milestone doubles as the [`DESKTOP_APP_PLAN.md`](../DESKTOP_APP_PLAN.md) P0 compute-feasibility spike.

```
    *Accept (the seamlessness guarantee):* a **parity test** ? the same clip through the
**WSL backend (Windows)**
    and the **`LinuxNativeRunner` (DO)** yields `Frame,Visibility,X,Y` CSVs that match
within tolerance **and**
    identical candidate windows; the WASB/TrackNet hybrid segmenter produces the same
rallies on DO as on the
    WSL box. (WASB and TrackNetV4 are torch vs TF ? keep them as separate runners/images
sharing `/scratch`;
    never co-install.)

    **M4 ? Scale-to-zero / cost-guard.**
    Confirm the endpoint scales workers to zero when idle; add a per-key rate limit + a
provider hard spend cap.
    *Accept:* after a burst, the dashboard shows **$0 idle GPU billing** between jobs; a
synthetic flood on one
    leaked key is throttled; the spend alert fires below $200 in a forced-overrun test.

    ## 5. Smallest first PR

    **PR: "Add bucket-aware I/O shim + `train`/`infer` dispatcher + (optional) `rally-torch`
Dockerfile ?
    CPU-clean path only."** In the indexer repo:
    1. A thin **storage shim**: `fetch(uri)?local_path` + `put(local_path, uri)` (over
`rclone`/`aws cp`),
       wired *behind* the existing `resolve_video_path`/`validate_input_path` so local paths
still work unchanged.
    2. A `run.sh` (or `python -m ?`) **dispatcher** (`mode ? {train, infer}`) calling the
existing CLI entry
       points ? **no pipeline-logic changes.**
    3. **De-hardcode** the `"output"` literals in
[`trajectory_hybrid.py:196,272`](../../backend/pipeline/segmenters/trajectory_hybrid.py)
       to honor an output root (small, isolated).
    4. *(Optional)* a `Dockerfile` for the torch image (pytorch CUDA 12.4 base, `apt
ffmpeg`, torch from the
       explicit `--index-url`, `supervision<0.30`, pre-baked COCO weights, `GEMINI_API_KEY`
via env).
    5. **CI:** run the **video-free regression subset** (on CPU, in a container or venv) ?
this *is* the M0 gate.

    This PR is intentionally WSL-free, GPU-optional, and touches no detector internals ? it
lands the
    container/bucket seam (the two things missing today) while deferring every HIGH-risk
component to a later,
    isolated PR.

    ## 6. Container image strategy (when you do containerize)

    - **Image A ? `rally-torch` (ship first; M0?M3):** base
`pytorch/pytorch:2.5.x-cuda12.4-cudnn9-runtime`;
      `apt ffmpeg`; torch/torchvision from `--index-url ?/cu124`; `supervision<0.30`; pre-
baked COCO weights;
      `GEMINI_API_KEY` as runtime secret. Covers gemini, motion, yolo_hybrid.
    - **Image B ? `rally-tracknet-tf` (M4+):** base `tensorflow/tensorflow:2.16-gpu`. This
image is what
      **kills the WSL bridge** ? inside Linux, TF-GPU runs natively. Lift the TrackNet/WASB
inference out of
      WSL into an in-container module reading frames from `/scratch`; **vendor** the
research repos + weights;
      replace hardcoded `device='cuda'` with the Linux-native `DetectorRunner` the ABC
already anticipates;
      keep the WSL bridge as one backend of a two-backend interface so Windows dev +
existing tests stay green.
      Torch and TF talk over a shared `/scratch` volume ? **never co-install TF + torch in
one image** (segfault).

```

12. user (2026-06-20T08:59:43.393Z)

```
1       # 03 · Platform options (Docker is optional)
2
3       **Status:** PROPOSAL · **Date:** 2026-06-20
4       **? All $ figures are approximate ? verify current provider terms before committing
budget.**
5
6       > You have **$200 in DigitalOcean** and are open to **GCS + a Cloud-Run-like**
architecture. This doc lays
7       > out the execution topologies (container *and* container-free), the provider trade-
offs, and a
8       > recommendation ? keeping the **side goal (exact per-video billing, [05](05-COST-
ATTRIBUTION.md))** front
9       > of mind, because it's the main thing that separates the options.
10
11      ## 1. Packaging: do you need Docker? (No ? but it helps in one place)
12
13      | Packaging | When it's right | Cost / effort |
14      |---|---|---|
15      | **Container (Docker/OCI)** | **Required** for serverless GPU (RunPod Serverless, Cloud
Run, Modal) and the cleanest reproducibility / "ship the exact image" story. | Two single-
framework images (torch; TF) once the WSL bridge is removed ? see [02](02-COMPUTE-MAP-AND-
CONTRACT.md). Highest portability. |
16      | **Plain VM + `venv`/`conda`** | A DO GPU Droplet or GCE VM you `git clone` + `pip
install -r requirements-train.txt` onto. Simplest first step; matches today's dev flow. | No
image to build, but **you** manage CUDA drivers + reproducibility is weaker. Fine for M1?M2. |
17      | **SkyPilot** (already named in
[`PLATFORM_ARCHITECTURE.md:156-160`](../PLATFORM_ARCHITECTURE.md)) | "No-Docker" managed
provisioning: a YAML spec launches your repo in a conda env on a spot VM on a backing cloud,
handles bucket mounts + auto-stop. | Backs GCP/AWS/Azure/Lambda/RunPod/Paperspace (**not** DO
Droplets natively). Great cost-control with zero image work. |
18      | **Managed (Modal / Vertex Custom Training)** | Lowest ops. Modal defines the image in
*Python* (not a raw Dockerfile); Vertex takes a container or a Python package. | Some lock-in;
Modal's bucket mount (`CloudBucketMount`) is convenient. |
19
20      **Verdict:** ship a container **where you go serverless** (M3 inference endpoint); a
**VM/SkyPilot venv**
21      path is perfectly good for M0?M2. Docker is *recommended, not required*.
22
23      ## 2. The bucket (decouple this from the compute choice)
24
25      Use **one S3-compatible bucket** so the same `rclone`/`aws` config works on any compute.
Two good picks:
26
27      - **DO Spaces** ? S3-compatible, ~$5/mo for 250 GB **with 1 TiB egress included** (the
best video-download
28        deal; egress is the line `HOSTING_PLAN.md` flags to watch). Natural with your DO
credits.
29      - **Cloudflare R2** ? **zero egress** fees; pairs with any compute. Best if reel-
download egress grows.
30      - **GCS** ? native if you go the Cloud Run / Vertex route; signed resumable uploads
(browser ? bucket).
31
32      **Mount discipline (decisive for video):** **sync-first** ? `rclone/aws cp` the whole
file to local NVMe
33      `/scratch` and decode *there*; do **not** FUSE-mount and random-seek over HTTP. Outputs
can stream back via
34      `rclone copy`.
35
36      ## 3. Provider matrix (for a single small GPU: T4/L4/A4000-class is plenty)
37
38      | Provider | "$200-ish" free credit? | BYO image / code | GPU $/hr (approx) | Scale-to-
zero / per-invocation metering | Bucket | Fit for this goal |
39      |---|---|---|---|---|---|---|
```

DigitalOcean	? **$200 ? confirmed applies to Paperspace GPUs** (+ Free Gradient plan = a **$0 M4000**, unbilled)	Droplet: VM + venv or Docker	GPU Droplets are **H100/L40S-class & pricey** (~$1.5?7/hr); **Paperspace/Gradient** (under DO) is cheaper ? **M4000 free**, then P4000/RTX4000/A4000 (~$0.51?0.76/hr)	? no true serverless GPU ? coarse (wall-clock) metering	**DO Spaces** (1 TiB egress incl.)	**Best for using the credits + storage; weakest for exact per-video billing.** Free M4000 makes the GPU smoke test $0; credit offsets faster cards.
Google Cloud	? **$300 / 90 days** (the only clean win that *exceeds* the ask)	Cloud Run (container) · Vertex Custom Training (container/pkg)	L4 on Cloud Run; Vertex Spot	? **Cloud Run GPU min-instances=0 ? exact per-invocation GPU-seconds**	GCS native	**Best for the side goal** (per-video billing = per-invocation bill) + train on Vertex Spot. IAM/quota friction.
RunPod	?? small (~$5, *not* $200)	**Ships your Docker image as-is**	L4 ~$0.39; RTX 4090 ~$0.34?0.69 (pod); serverless ~2?3×	? Serverless scale-to-zero, per-second	Network Volume / S3	Best literal "get an API key, run my image, drive the GPU" fit ? but no real free credit.
Modal	~$30/mo recurring	Image defined in Python	per-second GPU	? scale-to-zero	`CloudBucketMount`	Lowest ops; some lock-in.
SkyPilot ? (GCP/RunPod/Paperspace)	inherits backing cloud's	Repo in conda env (no Docker)	inherits	partial (managed auto-stop)	bucket mounts	Best no-Docker cost-control.

4. Recommendation (resolves the budget-vs-billing tension)

The tension: **DO has your $200 + cheap-egress storage but no serverless GPU**, while **GCP/RunPod give exact per-invocation metering** (which *is* your per-video bill) but the $200 isn't DO credit there.

Recommended phased path:

1. **M0 (local, $0):** CPU container *or* venv ? no provider.
2. **M1?M2 (train + manual runs):** **DO** ? spend the credits. Bucket = **DO Spaces**. Compute = a
   Paperspace/Gradient A4000 (or a small GPU Droplet) running the repo in a **venv** (no Docker needed
   yet) or your container. Train-from-bucket here is cheap (Gen-0 = minutes). Add a **start/stop script +
   hard spend cap** since DO won't scale-to-zero for you.
3. **M3 (inference endpoint + per-video billing):** stand the `infer` endpoint where **per-invocation
   metering is native** ? **GCP Cloud Run with an L4 GPU (min-instances=0)** on the $300 trial, *or*
   RunPod Serverless. Here a container *is* required, and the platform's per-job GPU-seconds become the
   per-video bill directly (see [05](05-COST-ATTRIBUTION.md)).

This uses the DO credits where they're strongest (compute + cheap-egress storage for the dev/train loop)
and the serverless platforms where they're strongest (metered, scale-to-zero inference). If you'd rather
stay single-vendor, GCP alone does everything (Vertex train + Cloud Run infer + GCS) on the $300 trial,
at the cost of IAM/quota setup.

DO is also the natural home for the M3.5 "WSL runs seamlessly" test ([04 §M3.5](04-BUILD-AND-TEST-PLAN.md)):
a **DO GPU Droplet** or **Paperspace/Gradient GPU** is **Ubuntu + NVIDIA + CUDA**, so you replicate the
`TRACKNET_WSL_SETUP.md` conda env **natively, minus WSL**, and prove the WASB/TrackNet detectors run off the
Windows box with byte-identical trajectory output. **Confirmed (DO support, 2026-06-20): the $200 credit
applies to Paperspace GPUs, and the Free Gradient plan includes an unbilled `M4000` GPU** ? so the parity

```
73      run + Gen-0 smoke can be **$0**, with the credit spent only on faster cards
(P4000/RTX4000/A4000) for
74      throughput. ? *The M4000 is Maxwell (`sm_52`): WASB's torch 1.11 supports it, but verify
your pinned
75      torch-2.x/CUDA build still ships `sm_52` (very old archs are occasionally dropped) ?
else fall back to the
76      VRAM heuristic on a P4000/RTX4000.*
77
78      ## 5. $200 burn-down (approximate ? Gen-0 + inference only; VLM excluded)
79
80      | Phase | GPU | Rate (approx) | Hours | Cost |
81      |---|---|---|---|---|
82      | M1 GPU smoke + M3.5 parity run | **free M4000** (Gradient plan) | **$0** | 20 |
**~$0** |
83      | Dev iteration / throughput | A4000 / RTX4000 | ~$0.51?0.76/hr | 15 | ~$9 |
84      | M2 Gen-0 train-from-bucket (minutes each, ~20 runs) | A4000 / L4 spot | ~$0.30/hr | 8
| ~$2 |
85      | M3 inference dev + ~200 burst `infer` jobs (~3 min GPU each) | L4 serverless |
~$0.80/hr eq. | ~10 GPU-hr | ~$8 |
86      | Storage + egress | DO Spaces / R2 | ? | ? | ~$3 |
87      | **Real burn** | | | **~$23** |
88      | Contingency / re-runs | | | | ~$30 |
89      | **Total** | | | | **well under $200** |
90
91      The budget is comfortable **precisely because the Gen-2 VLM is excluded.** *"Decoupled
train+infer on
92      $200"* is **true for the Gen-0 classifier + inference pipeline** and **unproven (likely
false) for the
93      owned VLM** ? don't conflate the two (see [06](06-RISKS-AND-OPEN-QUESTIONS.md)). **Set a
hard provider
94      spend cap at/under $200** so a runaway loop can't drain it.
95
```

13. user (2026-06-20T08:59:43.791Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16      from __future__ import annotations
17
18      import argparse
19      import csv
20      import json
21      import os
22      import sys
23      import tempfile
24      from typing import Any, List
25
26      from backend import storage
```

```
    from backend.config import load_config
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure
    from backend.utils.hashing import compute_video_id


    def _coerce(v: str) -> Any:
        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
        low = v.lower()
        if low in ("true", "false"):
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow([
                    f'{float(s.get("start_time", 0.0)):.3f}',
                    f'{float(s.get("end_time", 0.0)):.3f}',
                    s.get("shot_count", ""),
                    s.get("ending_reason", s.get("shot_count_confidence", "")),
                ])


    def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
        with open(path, "w") as f:
            json.dump({
                "video_id": video_id,
                "status": status,
                "segment_count": len(segments),
                "segments": [{"start": float(s.get("start_time", 0.0)),
                              "end": float(s.get("end_time", 0.0))} for s in segments],
            }, f, indent=2)


```

```
88      def cmd_infer(args: argparse.Namespace) -> int:
89          config = load_config(args.config) if args.config else load_config()
90          _apply_overrides(config, args.set)
91          storage_cfg = config.storage.model_dump()
92
93          scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
94          os.makedirs(scratch, exist_ok=True)
95          config.output.output_dir = scratch
96
97          # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
98          local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
99          print(f"[worker] pulled {args.video_uri} -> {local_video}")
100
101         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
102         video_id = compute_video_id(local_video)
103
104         # 3. VALIDATE (same registry as the main CLI).
105         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
106         failures = [r for r in val_results if not r.passed]
107         db = Database(os.path.join(scratch, config.output.db_name))
108         db.add_video(video_id, local_video, "failed" if failures else "processed",
109                      [r.model_dump() for r in val_results])
110
111         segments: List[dict] = []
112         segmenter_actual = None
113         segmentation_ran = False
114         status = "failed"
115         try:
116             if failures:
117                 print("[worker] validation FAILED: "
118                       + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
119                 return 2
120             seg_name = args.segmenter or config.indexing.default_segmenter
121             segmenter_cls = segmenter_registry.get(seg_name)
122             if not segmenter_cls:
123                 print(f"[worker] unknown segmenter: {seg_name!r} "
124                       f"(have {list(segmenter_registry.list_available())})")
125                 return 2
126             segmenter_actual = seg_name
127             result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
128             segmentation_ran = True
129             if isinstance(result, Failure):
130                 print(f"[worker] segmenter FAILED: {result.message}")
131                 return 3
132             segments = result.value if hasattr(result, "value") else result
133             status = "success"
134         finally:
135             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
136             emit_indexer_report(
137                 db=db, video_id=video_id, video_path=local_video, config=config,
138                 val_results=val_results, val_context=val_context,
139                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
140                 was_reingest=False, segmentation_ran=segmentation_ran,
141             )
142
143         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
144         out_local = os.path.join(scratch, "outputs", video_id)
145         os.makedirs(out_local, exist_ok=True)
146         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
```

```
147         _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
148
149         out_prefix = args.out_uri.rstrip("/")
150         pushed = []
151         for fn in sorted(os.listdir(out_local)):
152             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
153             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
154             pushed.append(uri)
155
156         print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
157         for u in pushed:
158             print("   ", u)
159         return 0
160
161
162     def _resolve_train_detector(args: argparse.Namespace, config: Any):
163         """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
164         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
165         stub=CPU mock). Returns the runner, or prints an error + returns None.
166
167         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
168         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
169         no shuttle detector and is rejected.
170         """
171         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
172
173         seg_cls = segmenter_registry.get(args.segmenter)
174         if not seg_cls:
175             print(f"[worker] unknown segmenter: {args.segmenter!r} "
176                   f"(have {list(segmenter_registry.list_available())})")
177             return None
178         seg = seg_cls(None, config)
179         if not isinstance(seg, TrajectoryHybridSegmenter):
180             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
181                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
182             return None
183         runner, _ = seg._resolve_runner(config.get("indexing", {}))
184         ok, msg = runner.healthcheck()
185         if not ok:
186             print(f"[worker] detector environment not ready: {msg}")
187             return None
188         print(f"[worker] detector ready ({args.segmenter}): {msg}")
189         return runner
190
191
192     def cmd_train(args: argparse.Namespace) -> int:
193         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
194         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
195
196         Two trajectory sources (the train pipeline + harness are identical either way):
197           * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
198             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
199           * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
200             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
```

```
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
                            if u.lower().endswith(".csv"))
        if len(label_uris) < 2:
            print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                  f"under {corpus}/labels/")
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
            name = fname.replace(".rallies.csv", "").replace(".csv", "")
            local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
                    continue
                local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
                                            config=storage_cfg)
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping (synth fixes them
at 30/1280).
                from backend.pipeline.segmenters.trajectory_hybrid import
TrajectoryHybridSegmenter
                fps, frame_width = TrajectoryHybridSegmenter._probe_fps_width(local_video)
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
```

```
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [sys.executable, "-m", "training.gen0.harness",
               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
        if args.floor:
            floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                           if "://" in args.floor else args.floor)
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(prog="python -m backend.worker",
                                    description="Storage-aware worker: pull ? run pipeline ?
push.")
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
        pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
        pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
        pi.add_argument("--config", default=None, help="config.json path (default:
```

```
./config.json)")
312         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
313         pi.add_argument("--max-frames", type=int, default=None)
314         pi.add_argument("--set", action="append", default=[], metavar="k=v",
315                         help="config override, e.g. --set indexing.skip_ai_handoff=true")
316         pi.set_defaults(func=cmd_infer)
317
318         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
319         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
320                         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
321         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
322         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
323         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
324                         help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
325                              "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
326                              "(needs videos/ in the corpus; honours detector_impl).")
327         pt.add_argument("--segmenter", default="wasb_hybrid",
328                         help="trajectory hybrid whose detector is used for --trajectory-
source detector")
329         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
330         pt.add_argument("--config", default=None)
331         pt.add_argument("--scratch", default=None)
332         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
333         pt.add_argument("--frame-width", type=float, default=1280.0,
334                         help="synth trajectory frame width (detector probes the video)")
335         pt.add_argument("--set", action="append", default=[], metavar="k=v")
336         pt.set_defaults(func=cmd_train)
337         return p
338
339
340     def main(argv=None) -> int:
341         args = build_parser().parse_args(argv)
342         return args.func(args)
343
344
345     if __name__ == "__main__":
346         sys.exit(main())
347
```

14. assistant (2026-06-20T08:59:46.007Z)

Now let me read the storage modules to map the StorageBackend contract precisely.

15. user (2026-06-20T08:59:46.967Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Drive
(follow-up).
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local URI is actually used (install the ``storage-s3`` /
``storage-gcs``
5       / ``storage-gdrive`` extras). The default ``local`` backend needs no third-party deps
and
6       preserves today's behaviour byte-for-byte.
7
```

```
8       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
9       ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
10      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
11      """
12      from __future__ import annotations
13
14      import os
15      from typing import Any, Dict, Iterator, Optional
16
17      from backend.storage.base import StorageBackend
18      from backend.storage.ref import StorageRef, StorageStat, parse_uri
19
20      __all__ = [
21          "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
22          "get_storage", "fetch", "put", "list_uris",
23      ]
24
25
26      def _backend_class(name: str):
27          """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
28          if name == "local":
29              from backend.storage.local import LocalStorage
30              return LocalStorage
31          if name == "s3":
32              from backend.storage.s3 import S3Storage
33              return S3Storage
34          if name == "gcs":
35              from backend.storage.gcs import GCSStorage
36              return GCSStorage
37          if name == "gdrive":
38              from backend.storage.gdrive import GDriveStorage
39              return GDriveStorage
40          raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
41
42
43      def get_storage(backend: Optional[str] = None, *,
44                      config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
45          """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
46          the implementation; its per-backend settings come from ``config[backend]`` merged
with
47          ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
48          cfg = config or {}
49          name = backend or cfg.get("backend", "local")
50          settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
51          settings.update(overrides)
52          return _backend_class(name)(**settings)
53
54
55      def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
56                config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
57          """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
58          (no copy); for s3/gcs it downloads to ``dst``."""
59          ref = parse_uri(uri)
60          return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
61
62
63      def put(local_path: "str | os.PathLike[str]", uri: str, *,
64              config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
```

```
65          """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
66          ref = parse_uri(uri)
67          get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
68          return uri
69
70
71      def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
72          """Yield the URI of every object under ``prefix_uri``."""
73          ref = parse_uri(prefix_uri)
74          for r in get_storage(ref.backend, config=config).list(ref):
75              yield r.uri
76
```

16. user (2026-06-20T08:59:47.908Z)

```
1       """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3       Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). Concrete backends
4       live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5       pulls ``boto3`` / ``google-cloud-storage`` / ``pydrive2``. See
``PLATFORM_ARCHITECTURE.md``
6       §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7       """
8       from __future__ import annotations
9
10      import os
11      from abc import ABC, abstractmethod
12      from typing import Iterator
13
14      from backend.storage.ref import StorageRef, StorageStat
15
16
17      class StorageBackend(ABC):
18          """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19          worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21          @abstractmethod
22          def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23                             *, overwrite: bool = False) -> str:
24              """Make the object available as a local file; return its local path. For the
25              ``local`` backend this is an identity passthrough (the object is already
local)."""
26
27          @abstractmethod
28          def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
29                  content_type: "str | None" = None) -> StorageRef:
30              """Upload/copy a local file to ``ref``; return ``ref``."""
31
32          @abstractmethod
33          def exists(self, ref: StorageRef) -> bool: ...
34
35          @abstractmethod
36          def stat(self, ref: StorageRef) -> StorageStat: ...
37
38          @abstractmethod
39          def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
40              """Yield a ``StorageRef`` per object under ``prefix``."""
41
42          @abstractmethod
43          def delete(self, ref: StorageRef) -> None: ...
44
```

```
45          def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
str:
46              """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
47              sign (``local`` cannot; Drive only emulates) ? callers must treat
48              ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
49              raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
50
```

17. user (2026-06-20T08:59:48.072Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<folderId>/<path>  -> gdrive   (follow-up backend)
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket or the Drive folder-id;
37          for ``local`` it is empty and ``key`` carries the filesystem path."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48      def parse_uri(uri: str) -> StorageRef:
49          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52          have no ``//``) is treated as a local filesystem path ? preserving today's
```

```
behaviour.
53          """
54          s = str(uri)
55          m = _SCHEME_RE.match(s)
56          if not m:
57              return StorageRef("local", "", s, f"local://{s}")
58          scheme = m.group(1).lower()
59          rest = s[m.end():]
60          if scheme == "gs":
61              scheme = "gcs"
62          if scheme == "local":
63              return StorageRef("local", "", rest, s)
64          if scheme in _KNOWN:
65              bucket, _, key = rest.partition("/")
66              return StorageRef(scheme, bucket, key, s)
67          raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
68
```

18. assistant (2026-06-20T08:59:50.911Z)

Now let me read the s3 and local backend implementations, the config storage model, and check the remaining docs (01 gap and 05/06 briefly for the single-box angle).

19. user (2026-06-20T08:59:51.842Z)

```
1       """S3 (and S3-compatible) storage backend ? first-class object buckets.
2
3       One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they
differ
4       only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily
so
5       ``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.
6
7       Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
8       style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` /
``AWS_SECRET_ACCESS_KEY``
9       / ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
10      (tests pass a fake; production lets the constructor build one via boto3).
11      """
12      from __future__ import annotations
13
14      import os
15      from typing import Any, Iterator, Optional
16
17      from backend.storage.base import StorageBackend
18      from backend.storage.ref import StorageRef, StorageStat
19
20
21      class S3Storage(StorageBackend):
22          def __init__(self, *, client: Any = None, endpoint_url: Optional[str] = None,
23                       region: Optional[str] = None, addressing_style: str = "auto",
24                       signed_url_ttl: int = 3600, **_ignored) -> None:
25              self._ttl = signed_url_ttl
26              if client is not None:
27                  self._client = client
28                  return
29              import boto3  # type: ignore  # lazy: only when an s3:// URI is actually used
30              from botocore.config import Config  # type: ignore
31              self._client = boto3.client(
32                  "s3",
33                  endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
34                  region_name=region,
35                  config=Config(s3={"addressing_style": addressing_style}),
36              )
37
```

```
        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            if dst is None:
                raise ValueError("S3Storage.fetch_to_local requires a dst path")
            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            self._client.download_file(ref.bucket, ref.key, dst)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            extra = {"ContentType": content_type} if content_type else None
            self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
            return ref

        def exists(self, ref) -> bool:
            try:
                self._client.head_object(Bucket=ref.bucket, Key=ref.key)
                return True
            except Exception:  # noqa: BLE001 - any boto/client error means "not found /
unreachable"
                return False

        def stat(self, ref) -> StorageStat:
            h = self._client.head_object(Bucket=ref.bucket, Key=ref.key)
            return StorageStat(size=int(h.get("ContentLength", 0)),
modified=h.get("LastModified"),
                               etag=h.get("ETag"), content_type=h.get("ContentType"))

        def list(self, prefix) -> Iterator[StorageRef]:
            paginator = self._client.get_paginator("list_objects_v2")
            for page in paginator.paginate(Bucket=prefix.bucket, Prefix=prefix.key):
                for obj in page.get("Contents", []) or []:
                    key = obj["Key"]
                    yield StorageRef("s3", prefix.bucket, key,
f"s3://{prefix.bucket}/{key}")

        def delete(self, ref) -> None:
            self._client.delete_object(Bucket=ref.bucket, Key=ref.key)

        def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
            op = "get_object" if method.upper() == "GET" else "put_object"
            return self._client.generate_presigned_url(
                op, Params={"Bucket": ref.bucket, "Key": ref.key}, ExpiresIn=ttl or
self._ttl)

```

20. user (2026-06-20T08:59:52.825Z)

```
    """Local-filesystem storage backend ? the default, zero new dependencies.

    Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
(the
    file is already local), ``put`` is a copy (a no-op when src == dst). With
``backend='local'``
    the whole pipeline behaves exactly as before the storage layer existed.
    """
    from __future__ import annotations

    import os
    import shutil
    from datetime import datetime
    from typing import Iterator

    from backend.storage.base import StorageBackend
```

```
15      from backend.storage.ref import StorageRef, StorageStat
16
17
18      class LocalStorage(StorageBackend):
19          def __init__(self, *, base_dir: str = "", **_ignored) -> None:
20              # Relative keys resolve under base_dir when set; absolute keys are used as-is.
21              self.base_dir = base_dir
22
23          def _path(self, ref: StorageRef) -> str:
24              p = ref.key
25              if self.base_dir and not os.path.isabs(p):
26                  return os.path.join(self.base_dir, p)
27              return p
28
29          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
30              # The object is already on the local FS ? return its path (identity). ``dst`` is
31              # honoured only if it names a *different* location (a copy is then made).
32              src = self._path(ref)
33              if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
34                  return src
35              dst = str(dst)
36              if os.path.exists(dst) and not overwrite:
37                  return dst
38              os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
39              shutil.copyfile(src, dst)
40              return dst
41
42          def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43              dst = self._path(ref)
44              if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45                  os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46                  shutil.copyfile(str(local_path), dst)
47              return ref
48
49          def exists(self, ref) -> bool:
50              return os.path.exists(self._path(ref))
51
52          def stat(self, ref) -> StorageStat:
53              st = os.stat(self._path(ref))
54              return StorageStat(size=st.st_size,
modified=datetime.fromtimestamp(st.st_mtime))
55
56          def list(self, prefix) -> Iterator[StorageRef]:
57              base = self._path(prefix)
58              if os.path.isdir(base):
59                  for root, _dirs, files in os.walk(base):
60                      for fn in sorted(files):
61                          full = os.path.join(root, fn)
62                          yield StorageRef("local", "", full, f"local://{full}")
63              elif os.path.exists(base):
64                  yield StorageRef("local", "", base, f"local://{base}")
65
66          def delete(self, ref) -> None:
67              path = self._path(ref)
68              if os.path.exists(path):
69                  os.remove(path)
70
```

21. user (2026-06-20T08:59:54.135Z)

```
4:Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
36:    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
57:    Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
63:    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
124:    # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
```

```
170:    # (logged + flagged on the window) so the degenerate single-window outcome is avoided
and the
186:    # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
228:    # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
234:    (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
266:    """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
276:    # mode). Default None preserves the single-user local 'any local file' workflow.
291:class StorageConfig(BaseModel):
292:    """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
293:    behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
294:    + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = Google
Drive
295:    (follow-up). Per-backend settings are loose dicts passed to the backend constructor ?
297:    (boto3/google auth chains supply credentials). See ``backend/storage/``."""
298:    backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
302:    s3: dict[str, Any] = Field(default_factory=dict)
303:    gcs: dict[str, Any] = Field(default_factory=dict)
```

22. assistant (2026-06-20T08:59:57.397Z)

Let me read the StorageConfig model fully and check the 01-gap doc and detector_impl seam, plus the stub_runner and detector base for the compute/runner contract.

23. user (2026-06-20T08:59:58.831Z)

```
285         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
286         max_upload_mb: int = 4096
287         max_part_mb: int = 95
288         allowed_upload_extensions: List[str] = ["mp4", "mov"]
289
290
291     class StorageConfig(BaseModel):
292         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
293         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
294         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = Google
Drive
295         (follow-up). Per-backend settings are loose dicts passed to the backend constructor
?
296         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
297         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
298         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
299         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
300         output_root: str = ""
301         local: dict[str, Any] = Field(default_factory=dict)
302         s3: dict[str, Any] = Field(default_factory=dict)
303         gcs: dict[str, Any] = Field(default_factory=dict)
304         gdrive: dict[str, Any] = Field(default_factory=dict)
305
306
307     class AppConfig(BaseModel):
308         """Root configuration object.
309
310         All runtime configuration should be accessed via an instance of this class
311         (or the loader) rather than raw dict lookups.
312         """
313
314         sport: Sport = "badminton"
315         ai_provider: str = "gemini"
316         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
317         # the alias never deprecates underneath us. Eval reports record the run date
```

```
alongside.
318         ai_model: str = "gemini-flash-latest"
319
320         validation: ValidationConfig = Field(default_factory=ValidationConfig)
321         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
322         output: OutputConfig = Field(default_factory=OutputConfig)
323         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
324         security: SecurityConfig = Field(default_factory=SecurityConfig)
```

24. user (2026-06-20T08:59:59.808Z)

```
1       # 01 · Existing plans for decoupling ? and the gap to the North Star
2
3       **Status:** research synthesis · **Date:** 2026-06-20
4
5       > The North Star is unusually well-anticipated **in design**. Across our strategy docs
we've already drawn
6       > nearly every seam the goal needs ? control/data-plane separation, a pluggable compute
backend, an
7       > object-storage abstraction, a train/serve wall, a rented-GPU provider menu, and a
budget framing. What
8       > we have **not** done is build them, package them into a container/VM image, wire them
to a real cloud
9       > GPU, or attach a cost meter. This doc inventories the plans, the current reality + its
blockers, and the
10      > precise gap.
11
12      ## 1. What already anticipates decoupling / cloud / storage / GPU-CPU split
13
14      ### 1a. Load-bearing architecture docs (genuinely toward the North Star)
15
16      - **[`PLATFORM_ARCHITECTURE.md`](../PLATFORM_ARCHITECTURE.md) ? the single most relevant
doc.** States the
17        principle outright: *"Decouple into layers with a clean control-plane vs data-plane
split ? the
18        orchestrator coordinates metadata and jobs; the heavy bytes and GPU work can live
elsewhere (even in the
19        user's own cloud)"* (`:71-74`). Defines the exact mechanism: a
**`ComputeBackend`/`ExecutionBackend`
20        registry** with `local | hosted | byo_worker` (`:143-148`), and the property that
makes lift-and-shift a
21        config change ? *"The pipeline code is unchanged; only where it's invoked and how it
gets bytes vary"*
22        (`:174`). Names the provisioning model (SkyPilot into the user's own cloud; Modal as
managed fallback,
23        `:156-160`) and recommends the retraining half run via `ms-swift` on *"a rented GPU
(RunPod / Lambda /
24        Modal / SkyPilot-to-your-cloud)"* dropping into the **ComputeBackend / BYO-compute**
seam (`:369-373`).
25        Defines the **`StorageBackend`** over `fsspec`/`smart_open` with S3/GCS/Azure backends
+ presigned
26        browser?bucket transfers (`:124-135`), unifies train + inference as Compute jobs over
a Storage corpus
27        (`:306-312`), and specifies an agent/CI-runnable one-command **`train_speedrun`**
(`:390-419`). It even
28        states the durability requirements any remote-GPU layer must satisfy: **idempotency,
resumability** (*"a
29        40-min GPU job mustn't restart on a worker blip"*), progress states (`:241-242`).
30      - **[`HOSTING_PLAN.md`](../HOSTING_PLAN.md) ? the concrete cloud topology + budget
framing.** Beta design
31        separates a **scale-to-zero Cloud Run control plane** from a **GPU box acting as a
pull-worker**, with
32        uploads going *"browser ? GCS directly"* so *"the box pulls only what it processes"*
(`:55-61`); lays out
33        a post-credits envelope (~$15?20/mo) on **GCP credits** (`:64-67`) ? but note this
```

covers only the
control plane, **not GPU compute**.

1b. Roadmap / work-item docs (compute-split anticipated; cloud half partial)

- **[`NEXT_STEPS.md`](../NEXT_STEPS.md)** ? where the GPU-vs-CPU split is *operationalized today*: a GPU box runs `fusion_golden` to extract `*_golden_features.json`, hands them off via a shared cloud folder, and a separate **CPU** session runs `fusion_compare`. A working decoupling ? but of two *local* machines over a OneDrive folder, not a rented cloud GPU. Also ships the portable one-command Gen-0 harness.
- **[`BACKLOG.md`](../BACKLOG.md)** ? the only place the rented cloud GPU is named as a *cost* question: *"Quantify cost-to-Gen-2 (the largest open risk) ? benchmark real VRAM/token/$ for the VLM SFT+GRPO on a rented A100/L40S ? before standing up any owned serving stack"* and *"Real GPU $/video & latency ? on the target VM before pricing."*
- **[`ROADMAP.md`](../ROADMAP.md)** ? motivation (replace paid Gemini with a free, offline, retrainable segmenter) but aimed at *local/air-gapped* execution ? adjacent, not cloud-bound.
- **`POST_166_?RISK_ASSESSMENT.md`** ? repeatedly isolates *"GPU extraction over the full corpus"* as *"the hardware-gated step"* ? the clean seam for relocating just the GPU work.

1c. Owned-model / experiment-infra docs (strongest on the compute seams)

- **`archives/research/OWNED_MODEL_TRAINING_STUDY.md`** ? closest thing to a rented-GPU procurement plan: ranks providers with prices (*"Modal ? PRIMARY ? RunPod (Secure Cloud tier ONLY) ? FALLBACK ? Self-hosted RTX 2070 Super ? Gen-0 training + ALL inference + CI only. Cannot QLoRA a 7B video VLM"*) and bounds cost (*"Gen-0: ~$0 (local 2070 Super) ? Gen-2 LoRA: ~$5?40/run on a rented 24?48 GB GPU ? 24 GB is a floor"*).
- **[`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](../OWNED_MODEL_IMPLEMENTATION_PLAN.md)** ? pins Gen-2 VLM training to *"Cloud train on Modal (no-access) / RunPod-Secure"*; notes heavy compute already runs *"out-of-process (ffmpeg, WASB-in-WSL, cloud GPU per-job)."*
- **[`training/README.md`](../../training/README.md)** ? the decoupling seam that matters most: the **ADR-008 train/serve wall**, where *"the boundary ? is an artifact bundle (weights + manifest), never a code import,"* pre-staged to split into its own repo when *"cloud training"* triggers fire; training-only deps isolated in `requirements-train.txt`.
- **`EXECUTION_POLICY.md`** ? explicitly anticipates *"a future remote compute backend"* under one wait/resume/abort contract over a master-less, self-scheduling job table (*"the DB is the coordination"*).
- **`EXPERIMENT_HARNESS.md`** ? implements the GPU/CPU split on the inference side: run detection once on GPU, persist, then *"re-score any number of variants OFFLINE ? deterministically, with no GPU."*

1d. Storage / locality docs (the data-plane scaffolding)

- **`VIDEO_LOCALITY_MODEL.md`** ? reframes the problem (*"any design that moves the 10 GB is moving the wrong thing ? the smallest artifact that must cross the network"*), builds an L0?L4 locality ladder where the ComputeBackend split *"falls out naturally,"* and names the **unsolved** *"Key

custody ? client calls
74   Gemini with whose key?"* ? the closest existing engagement with **API-key-gated
compute**.
75   - **`STORAGE_SHARING_MODEL.md`** ? makes *"Compute (the cost)"* a first-class axis with
"Their GPU" as an
76   end-state mapped to PLATFORM P3, and notes the licensing prerequisite for off-provider
execution.
77   - **[`GOLDEN_DATA_SHARING.md`](../GOLDEN_DATA_SHARING.md)** ? the only place the GPU/CPU
decoupling is
78   *implemented and container-aware*: GPU-only extraction vs *"pure-CPU re-scoring ? any
session (incl. a
79   CPU dev container) can run them,"* with `RALLY_GOLDEN_DIR` path indirection and a
**committed video-free
80   regression subset**.
81
82   ## 2. Current reality ? and which parts are blockers to a portable cloud GPU
83
84 | Today | Blocker to a rented Linux GPU container? |
85 |---|---|
86 | Heavy detectors run **only** through a **WSL2 bridge** (`wsl -d Ubuntu ? conda
activate`), hardcoded `~/models` paths, hardcoded `device='cuda'` (no CPU fallback), **un-
vendored** research repos (WASB, TrackNet), **conflicting TF-vs-torch** runtimes | **HIGH.**
This is the single load-bearing portability risk. *Upside:* on a Linux container the WSL premise
disappears ? TF runs natively. The `DetectorRunner` ABC
([`detectors/base.py`](../../backend/pipeline/detectors/base.py)) already anticipates the swap,
but the native implementation **does not exist** and has **never been run**. |
87 | Producer?trainer handoff is **same-account OneDrive auto-sync**
(`GOLDEN_DATA_SHARING.md:14-16`) | **HIGH.** A proto-bucket, not an API-key/signed-URL S3/GCS/R2
bucket. Won't work for a rented GPU on a different account. |
88 | Everything assumes a **seekable local file**: `cv2.VideoCapture`, `ffmpeg -i <path>`,
whole-file MD5, an `allowed_video_root` path check ([`main.py`](../../backend/main.py)) |
MEDIUM. No URL/stream path. Needs a download-to-local shim + output upload. (Sync-first copy
to NVMe is the right shape.) |
89 | **Hardcoded literal `"output"`** in
[`trajectory_hybrid.py:196,272`](../../backend/pipeline/segmenters/trajectory_hybrid.py) ?
`--output-dir` does not redirect these | **MEDIUM ? confirmed live 2026-06-20.** Jobs aren't yet
relocatable to a mounted/bucket path. |
90 | `torch>=2.12.0` pinned with no install index; CPU-vs-CUDA wheel needs an out-of-band
`--index-url` | **LOW.** The Dockerfile/installer must choose `cu124` vs `cpu` explicitly. |
91 | Default segmenter is **Gemini (paid API)**, slated for *removal* (`ROADMAP.md`); the
only API key in the system is the Gemini key | **CONTEXT.** The "API key that drives the GPU"
the North Star wants **does not exist** anywhere ? only the Gemini key, which we're trying to
retire. |
92 | Collector keys by human `video_id`; export manifest **omits `md5`** | **MEDIUM.** The
MD5-keyed consumer needs it once videos arrive from a bucket (see [02](02-COMPUTE-MAP-AND-
CONTRACT.md)). |
93
94   The **producer stage** ([`sports-data-collector`](../../../sports-data-collector)) is
CPU-first and *"the
95   gap is narrow"*: its only GPU touchpoint is opt-in `nvenc` (defaults to CPU `libx264`);
it shares the same
96   local-filesystem + MD5 assumptions.
97
98   ## 3. The gap ? what has *not* been planned
99
100   1. **Zero containerization, anywhere.** No doc mentions a Dockerfile, image, or
"build/test inside a
101   container." "In-container" in the roadmap refers to the dev/agent *sandbox*, not a
deployable GPU image.
102   The North Star's central artifact ? a container (or a provisioned VM) that trains and
infers ? has no
103   base-image choice, no TF/torch-split decision, no `train`-vs-`infer` dispatcher.
104   2. **No concrete rented-GPU provisioning recipe or API-key flow.** Providers are *named*
and even priced,
105   but there is no *chosen* provider, no API-key/credential flow to spin up a GPU, no

```
machine-type/region/
106        quota plan, and **no notion of an API key that drives the GPU itself** (serverless
`Authorization:
107        Bearer <KEY>` endpoints are entirely absent from the plans).
108     3. **The GPU-vs-CPU split isn't articulated *for rented hardware*, and the $200 envelope
is unvalidated.**
109        "Cost-to-Gen-2" is repeatedly named *"the largest open risk, currently
unquantified"*; the 6?8-GPU
110        ms-swift GRPO reference gives direct reason to doubt the heavy retrain path fits
~$200. No doc sizes a
111        full train+infer cycle against the budget. (This directory does ? see
[03](03-PLATFORM-OPTIONS.md) ? by
112        **excluding** the VLM.)
113     4. **The bucket-fed flywheel and the backends themselves are roadmap, not built.** The
training corpus
114        *"lives in Storage"* but nothing watches a bucket of new videos to launch a retrain
(flywheel is P5,
115        deferred). `StorageBackend`/`ComputeBackend` are P2/P3 future work; today execution
is a single-worker
116        `ThreadPoolExecutor` on the owner's Windows box. **Neither half ? "retrain from a
bucket" nor "inference
117        on new uploads" ? has an end-to-end storage path.** No remote inference endpoint is
designed; the
118        orchestration/queue tech for remote workers is undecided (Arq vs Postgres-queue vs
Hatchet vs Temporal).
119     5. **No cost meter.** Nothing records GPU-seconds, egress, or per-job cost; the `jobs`
table has no cost
120        columns. The side goal (per-video / per-training billing for a non-owner) is wholly
net-new ? see
121        [05](05-COST-ATTRIBUTION.md).
122
123     **Three further constraints sit beside the gap** (detailed in [06](06-RISKS-AND-OPEN-
QUESTIONS.md)): the
124     ratified posture is *"rent nothing until the owned model has a ship-gate-passing
reason"* and the harness is
125     *"single-tenant / local-first ? explicitly NOT a service"* (the North Star **requires
overriding both**);
126     **DPA / no-train legal gating** limits which videos may go to cloud (*"only
owned/consented footage, never
127     user uploads"*) ? which may *legally* gate the "inference on new uploads" half; and per-
checkpoint
128     **weight/dataset licensing** (WASB-C3 provenance) gates shipping any cloud-trained model
commercially.
129
130     **Net:** we have the right *seams* ? control/data-plane split, a named ComputeBackend
registry, an
131     artifact-bundle train/serve wall, an implemented GPU-extract/CPU-rescore split, a
portable one-command
132     training harness, content-addressed MD5 identity, a StorageRef abstraction, and a priced
provider menu.
133     What's missing is everything that makes them the North Star: **a container/VM image
(likely a two-image
134     TF/torch split that deletes the WSL bridge), a chosen provider with an API-key/scale-to-
zero provisioning
135     flow, a real bucket replacing OneDrive, a download-to-local shim + output upload, path
de-hardcoding, a
136     bucket-triggered retrain, a remote inference endpoint, a cost meter, and a validated
burn-down proving a
137     train+infer cycle fits ~$200.**
138
```

25. user (2026-06-20T09:00:01.426Z)

```
docs/DECOUPLED_COMPUTE/01-EXISTING-PLANS-AND-GAP.md:125:*"single-tenant / local-first ?
explicitly NOT a service"* (the North Star **requires overriding both**);
```

```
docs/DECOUPLED_COMPUTE/05-COST-ATTRIBUTION.md:63:The **`pricebook` is seeded from the same
provider-grounded GPU set as the device profiles**
docs/DECOUPLED_COMPUTE/06-RISKS-AND-OPEN-QUESTIONS.md:45:  ship-gate-passing reason"*
(`VIDEO_LOCALITY_MODEL.md`), and the harness is *"single-tenant / local-first
docs/DECOUPLED_COMPUTE/07-COMPUTE-ABSTRACTION.md:96:    def detect(cls, config) ->
"DeviceContext":   # probe spec ? resolve profile (§5)
docs/DECOUPLED_COMPUTE/07-COMPUTE-ABSTRACTION.md:127:2. **Matched named profile** ? e.g. an
`L4_24GB` profile (bf16, batch N, chunk T).
docs/DECOUPLED_COMPUTE/07-COMPUTE-ABSTRACTION.md:135:> profiles only for GPUs your providers
actually rent, and fall back to a **VRAM heuristic** for everything
docs/DECOUPLED_COMPUTE/HANDOFF.md:52:1. **GPU box** (owner): spin up a DO GPU Droplet **in
`sgp1`** (co-located with the bucket) ? `SETUP_NEW_MACHINE.md` §2
docs/PLATFORM_ARCHITECTURE.md:60:| Execution | **async job model shipped** ? `/api/process`
returns **202 + job_id**, a single-worker `ThreadPoolExecutor` drains the `jobs` table, `GET
/api/jobs/{id}` polls status | single-tenant + local-only; can't dispatch to a user's VM/GPU yet
|
docs/PLATFORM_ARCHITECTURE.md:227:that first, single-tenant, then generalize.
docs/PLATFORM_ARCHITECTURE.md:244:    Postgres-backed queue + worker** ? likely sufficient for
the *first* single-tenant
docs/PLATFORM_ARCHITECTURE.md:475:| **P0** | ? **DONE ? Async job model, single-tenant**
(DEFERRED #16) | `jobs` table via `user_version` ladder; single-worker `ThreadPoolExecutor`;
`/api/process` ? 202 + job_id; `GET /api/jobs/{id}` status |
```

26. assistant (2026-06-20T09:00:06.098Z)

I now have a thorough picture of the storage contract, the worker CLI, the compute abstraction, what's built vs next, and the single-box mapping. Note "single-box profile" is not a literal named term in these docs ? the docs describe the general decoupled model where the single-box case is the degenerate config (`backend=local`, worker in-process). Let me verify two remaining details: the ComputeBackend's current existence (planned vs built) and the detector_impl resolution seam, to ground the single-box claims precisely.

27. user (2026-06-20T09:00:08.929Z)

```
=== ComputeBackend in code? ===
=== detector_impl seam ===
61:            f"detector_impl='native' is not supported for the "
63:            f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
65:    def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
68:        ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
76:        impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
79:            logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
81:            return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
83:            logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
190:        # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
191:        # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
=== stub_runner exports ===
38:class StubConfig:
50:    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "StubConfig":
64:class StubDetectorRunner(DetectorRunner):
67:    def __init__(self, cfg: Optional[StubConfig] = None):
70:    def healthcheck(self) -> Tuple[bool, str]:
73:    def _rows(self) -> List[Tuple[int, int, float, float]]:
97:    def run_predict(self, video_win_path: str, output_win_dir: str,
124:def synth_trajectory_from_labels(labels_csv: str, out_csv: str, *, fps: float = 30.0,
```

28. assistant (2026-06-20T09:00:12.240Z)

This confirms: `ComputeBackend` is not in code (planned only); the `detector_impl` seam already supports `auto`(WSL) / `stub`(CPU mock) / `native`(rejected with a clear error ? the M3.5 NativeWasbRunner doesn't exist yet). Let me read the `_resolve_runner` block to quote it

precisely.

29. user (2026-06-20T09:00:15.575Z)

```
55              raise NotImplementedError
56
57          def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
58              """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
59              native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
60              raise NotImplementedError(
61                  f"detector_impl='native' is not supported for the "
62                  f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
63                  f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
64
65          def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
66              """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
67
68              ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
69              ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
70              is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
71              in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
72              ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
73              (the WSL runner) so the default production path is unchanged
74              (``docs/DECOUPLED_COMPUTE/``).
75              """
76              impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77              if impl == "stub":
78                  from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79                  logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
80                              self.display_label or "trajectory")
81                  return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82              if impl in ("native", "native_wasb", "wasb_native"):
83                  logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
84                              self.display_label or "trajectory")
85                  return self._build_native_runner(idx_cfg)
86              return self._build_runner(idx_cfg)
87
88          @staticmethod
89          def _probe_fps_width(video_path: str) -> tuple:
90              """Return (fps, frame_width) for a video, with sensible fallbacks."""
91              cap = cv2.VideoCapture(video_path)
92              fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
93              width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
94              cap.release()
95              return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
96
97          @staticmethod
98          def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
99              """Provider-reported exchange count, else a duration-based estimate.
```

30. assistant (2026-06-20T09:00:19.022Z)

I have everything I need. The `native` branch routes to `_build_native_runner`, which the base class refuses (`trajectory_hybrid.py:60-63`) until a WASB subclass overrides it ? confirming M3.5 (NativeWasbRunner) is the unbuilt next piece. Now I'll return the structured findings.

31. user (2026-06-20T09:01:44.796Z)

Output does not match required schema: root: must have required property 'worker_cli', root: must have required property 'compute_abstraction', root: must have required property 'built_vs_next', root: must have required property 'single_box_fit', root: must have required property 'file_refs'

32. assistant (2026-06-20T09:01:48.215Z)

The schema validation failed because I used `<parameter name=...>` for subsequent fields but the first one used a different key ? let me re-call with all fields properly formed.

33. user (2026-06-20T09:02:55.570Z)

Structured output provided successfully

34. assistant (2026-06-20T09:02:57.714Z)

Structured output delivered successfully.